

Backend New Routes Test Guide

Parking Building Management System

Swagger checklist for all newly implemented backend routes: roles, building/floor/slot, monthly passes, slot registration with VNPay, parking sessions, checkout fees and penalties.

Generated for repository backend/SWP-The-Second-Project-Backend
Use after applying db/schema.sql and starting npm run dev.

0. Preconditions

1. Run database schema. Open MySQL and run db/schema.sql. If an ALTER TABLE line fails because a column already exists, skip that duplicate ALTER and continue.
2. Update .env for VNPay sandbox if testing VNPay flows.

```
VNPAY_TMN_CODE=your_sandbox_tmn_code
VNPAY_HASH_SECRET=your_sandbox_hash_secret
VNPAY_PAYMENT_URL=https://sandbox.vnpayment.vn/paymentv2/vpcpay.html
VNPAY_RETURN_URL=http://localhost:5000/api/payments/vnpay-return
```

3. Seed users and vehicles.

```
npm run seed:users
npm run seed:vehicles
```

4. Start backend.

```
npm run dev
```

5. Open Swagger UI.

```
http://localhost:5000/api-docs
```

NOTE: Known test accounts after seed: admin@test.com / 123456, manager@test.com / 123456, staff@test.com / 123456, user1@test.com / 123456, user2@test.com / 123456.

1. Auth and Role Tokens

1.1 Login admin

6. Swagger > Auth > POST /api/auth/login

```
{
  "emailOrPhone": "admin@test.com",
  "password": "123456"
}
```

7. Copy data.token. Click Authorize and enter: Bearer <admin_token>.
8. Expected: response success=true, user.role=ADMIN.

1.2 Login parking manager

9. POST /api/auth/login

```
{
  "emailOrPhone": "manager@test.com",
  "password": "123456"
}
```

10. Authorize with Bearer <manager_token>.
11. Expected: user.role=PARKING_MANAGER.

1.3 Login parking staff

12. POST /api/auth/login

```
{
  "emailOrPhone": "staff@test.com",
```

```
"password": "123456"
}
```

13. Authorize with Bearer <staff_token>.

14. Expected: user.role=PARKING_STAFF.

1.4 Login registered user

15. POST /api/auth/login

```
{
  "emailOrPhone": "user1@test.com",
  "password": "123456"
}
```

16. Authorize with Bearer <user_token>.

17. Expected: user.role=USER.

2. Admin User and Role Routes

NOTE: Use ADMIN token for this section. These routes verify role management added for business roles.

18. GET /api/users/roles

19. Expected: accountRoles includes ADMIN, PARKING_MANAGER, PARKING_STAFF, USER.
businessRoles also lists WALK_IN_GUEST as non-account role.

20. GET /api/users

21. Expected: list of seeded users including admin, manager, staff, and normal users.

22. GET /api/users/{id}

23. Use a real user id from GET /api/users. Expected: user detail with role and building info.

24. PATCH /api/users/{id}/role

```
{
  "role": "PARKING_MANAGER"
}
```

25. Expected: role updated. Important: that user must login again to receive a JWT with the new role.

26. Negative test: call PATCH /api/users/{id}/role with USER token. Expected: 403.

3. Parking Manager - Building CRUD

NOTE: Use PARKING_MANAGER token. Admin token should not be accepted for manager-only CRUD unless code is changed later.

27. POST /api/buildings

```
{
  "name": "Building Test A",
  "address": "Thu Duc, HCMC"
}
```

28. Expected: 201. Save data.id as buildingId.

29. GET /api/buildings

30. Expected: list includes Building Test A and count fields: floorCount, motorbikeFloorCount, carFloorCount, carSlotCount.

31. GET /api/buildings/{id}

32. Expected: detail of created building.

33. PATCH /api/buildings/{id}

```
{
  "name": "Building Test A Updated",
  "address": "District 9, HCMC"
}
```

34. Expected: name/address updated.

35. DELETE /api/buildings/{id}
36. Use only on a temporary building with no test data you still need. Expected: deleted id returned. For the rest of this guide, create/keep one building.

4. Parking Manager - Floors

NOTE: Use PARKING_MANAGER token. Floors belong to a building. Motorbike floors use capacity. Car floors use slots.

4.1 Create motorbike floor

37. POST /api/buildings/{buildingId}/floors

```
{
  "name": "B1 - Motorbike",
  "floorType": "MOTORBIKE",
  "capacity": 300,
  "status": "ACTIVE",
  "note": "Motorbike capacity floor"
}
```

38. Expected: 201. Save data.id as motorbikeFloorId. capacity=300, currentCount=0.

4.2 Create car floor

39. POST /api/buildings/{buildingId}/floors

```
{
  "name": "B2 - Car",
  "floorType": "CAR",
  "status": "ACTIVE",
  "note": "Car slot floor"
}
```

40. Expected: 201. Save data.id as carFloorId. capacity is null.

4.3 List/detail/update/delete floors

41. GET /api/buildings/{buildingId}/floors
42. Expected: returns both motorbike and car floors. Car floor has slotCount fields.
43. GET /api/floors/{id}
44. Expected: floor detail.
45. PATCH /api/floors/{id}

```
{
  "status": "MAINTENANCE",
  "note": "Temporarily locked for repair"
}
```

46. Expected: status updated. Set it back to ACTIVE before check-in tests.

```
{
  "status": "ACTIVE"
}
```

47. DELETE /api/floors/{id}
48. Expected: floor deleted. Use this only for temporary test floors.

4.4 Floor negative tests

49. Create MOTORBIKE floor without capacity. Expected: 400.
50. Change a CAR floor with existing slots to MOTORBIKE. Expected: 400 until slots are deleted.

5. Parking Manager - Car Slots

NOTE: Use PARKING_MANAGER token. Slots can only be created under CAR floors.

51. POST /api/floors/{carFloorId}/slots

```
{
```

```

    "slotCode": "C-01",
    "status": "AVAILABLE",
    "sizeLabel": "STANDARD",
    "positionDescription": "Near gate",
    "note": "First test slot"
  }

```

52. Expected: 201. Save data.id as slotId.

53. Create more slots for later tests: C-02 and C-03 with status AVAILABLE.

54. GET /api/floors/{carFloorId}/slots

55. Expected: list of slots for the car floor.

56. GET /api/slots/{id}

57. Expected: slot detail with floor/building info.

58. PATCH /api/slots/{id}

```

{
  "status": "MAINTENANCE",
  "note": "Testing maintenance status"
}

```

59. Expected: status updated. Set back to AVAILABLE before session tests.

```

{
  "status": "AVAILABLE"
}

```

60. DELETE /api/slots/{id}

61. Expected: slot deleted. Use this only for temporary slot, not the main test slot.

62. Negative test: POST /api/floors/{motorbikeFloorId}/slots. Expected: 400 because motorbike floors use capacity, not slots.

6. Admin + User Vehicle Prerequisites

NOTE: Some new routes require an approved CAR or MOTORBIKE. Use existing vehicle ids from seed or create new ones.

63. Authorize as USER. POST /api/vehicles

```

{
  "plateNumber": "59A-TEST99",
  "vehicleType": "CAR",
  "brand": "Toyota",
  "color": "White",
  "buildingId": 1
}

```

64. Expected: vehicle status PENDING. Save vehicleId.

65. Authorize as ADMIN. PATCH /api/vehicles/{vehicleId}/approve

66. Expected: vehicle status APPROVED.

67. Repeat with a MOTORBIKE if you need monthly pass tests for motorbike.

7. Parking Manager - Monthly Passes

NOTE: Use PARKING_MANAGER token. This manually creates monthly passes for testing gate flow. Slot registration payment can also act as an active car monthly slot registration.

68. POST /api/monthly-passes

```

{
  "vehicleId": 3,
  "amount": 120000,
  "startDate": "2026-06-11",
  "endDate": "2026-07-11",
  "note": "Monthly pass test"
}

```

69. Expected: 201. Save monthlyPassId.

- 70. GET /api/monthly-passes
- 71. Expected: list includes the new pass.
- 72. GET /api/monthly-passes/{id}
- 73. Expected: monthly pass detail.
- 74. Negative test: create monthly pass for PENDING/REJECTED vehicle. Expected: 400.

8. User - Slot Registration + VNPay Sandbox

NOTE: Use USER token. This flow registers an approved CAR into an available CAR slot and creates a VNPay sandbox payment URL.

- 75. POST /api/slot-registrations

```
{
  "vehicleId": 3,
  "slotId": 1,
  "amount": 100000,
  "bankCode": "NCB",
  "locale": "vn",
  "startDate": "2026-06-11",
  "endDate": "2026-07-11",
  "note": "Register car slot with VNPay sandbox"
}
```

- 76. Expected: 201. Response contains registration and payment.paymentUrl. Slot becomes RESERVED.
- 77. Open payment.paymentUrl in browser and complete VNPay sandbox payment.
- 78. VNPay redirects to GET /api/payments/vnpay-return. Expected after success: payment SUCCESS, registration PAID.
- 79. GET /api/slot-registrations/my
- 80. Expected: list includes the registration.
- 81. GET /api/slot-registrations/{id}
- 82. Expected: detail of your registration.
- 83. Negative tests: use MOTORBIKE vehicle, unapproved vehicle, non-owned vehicle, or unavailable slot. Expected: 400/404.

9. Payment Callback Routes

NOTE: Usually these are hit by VNPay. Manual Swagger execution is hard because it requires a valid vnp_SecureHash.

- 84. GET /api/payments/vnpay-return
- 85. Expected when called by VNPay with valid query: backend verifies checksum and updates payment/result.
- 86. GET /api/payments/vnpay-ipn
- 87. Expected when called by VNPay with valid query: returns JSON with RspCode and Message.
- 88. Negative test: call either callback without valid VNPay params. Expected: invalid checksum or VNPay-style RspCode 97 for IPN.

10. Parking Staff - Active Sessions

NOTE: Use PARKING_STAFF token for all parking session routes.

- 89. GET /api/parking-sessions/active
- 90. Expected: active and pending-payment sessions. Initially may be empty.

11. Parking Staff - Walk-in Motorbike Flow

- 91. POST /api/parking-sessions/check-in
- ```
{
```

```
 "plateNumber": "59M-WALK01",
 "vehicleType": "MOTORBIKE",
 "buildingId": 1,
 "note": "Walk-in motorbike"
}
```

92. Expected: 201. pricingType=TURN, customerType=WALK\_IN\_GUEST, status=ACTIVE.  
Motorbike floor currentCount increases by 1. Save sessionId.

93. POST /api/parking-sessions/{sessionId}/check-out

```
{
 "violationFee": 0,
 "paymentMethod": "CASH",
 "paidNote": "Staff collected cash"
}
```

94. Expected: baseFee=4000, totalAmount=4000, status=COMPLETED, paymentStatus=PAID.  
Motorbike floor currentCount decreases by 1.

95. Repeat checkout with violationFee=50000. Expected: totalAmount=54000.

## 12. Parking Staff - Walk-in Car Flow

96. POST /api/parking-sessions/check-in

```
{
 "plateNumber": "59A-WALK01",
 "vehicleType": "CAR",
 "slotId": 2,
 "note": "Walk-in car"
}
```

97. Expected: 201. pricingType=HOURLY, customerType=WALK\_IN\_GUEST, slot status becomes OCCUPIED. Save sessionId.

98. POST /api/parking-sessions/{sessionId}/check-out

```
{
 "violationFee": 0,
 "paymentMethod": "CARD",
 "paidNote": "Staff confirmed card payment"
}
```

99. Expected: baseFee at least 20000, totalAmount=baseFee, status=COMPLETED, slot returns AVAILABLE.

100. VNPay checkout example:

```
{
 "violationFee": 50000,
 "violationNote": "Parked in wrong area",
 "paymentMethod": "VNPAY",
 "bankCode": "NCB",
 "locale": "vn"
}
```

101. Expected: response contains payment.paymentUrl and session status PENDING\_PAYMENT.  
After VNPay success, session becomes COMPLETED and slot returns AVAILABLE.

## 13. Parking Staff - Registered User With Monthly Pass

102. Create/ensure an APPROVED vehicle has an active monthly pass or paid slot registration.

103. POST /api/parking-sessions/check-in with that vehicle plate.

```
{
 "plateNumber": "59A-12345",
 "vehicleType": "CAR",
 "slotId": 1,
 "note": "Registered car with monthly pass"
}
```

104. Expected: pricingType=MONTHLY\_PASS, customerType=REGISTERED\_USER, base fee will be 0 at checkout.

105. Checkout with no violation:

```
{
 "violationFee": 0
}
```

106. Expected: totalAmount=0, paymentMethod=MONTHLY\_PASS, status=COMPLETED.

107. Checkout with violation:

```
{
 "violationFee": 50000,
 "violationNote": "Violation still charged despite monthly pass",
 "paymentMethod": "CASH"
}
```

108. Expected: baseFee=0, violationFee=50000, totalAmount=50000, status=COMPLETED.

## 14. Permission Matrix Tests

109. USER calls POST /api/buildings. Expected: 403.

110. ADMIN calls POST /api/buildings. Expected: 403 with current role design; manager owns building/floor/slot config.

111. PARKING\_MANAGER calls POST /api/parking-sessions/check-in. Expected: 403.

112. PARKING\_STAFF calls GET /api/users. Expected: 403.

113. No token calls protected route. Expected: 401.

## 15. Final E2E Demo Flow

114. Admin login and verify roles/users.

115. Manager creates building, motorbike floor, car floor, and several available slots.

116. User creates a CAR vehicle; admin approves it.

117. Manager creates a monthly pass OR user registers a slot and pays by VNPay sandbox.

118. Staff check-in registered car. If monthly pass/paid slot is valid, base fee is 0.

119. Staff check-out car with violationFee to prove penalties still apply.

120. Staff check-in walk-in motorbike and checkout cash: expected 4000 VND.

121. Staff check-in walk-in car and checkout card/VNPay: expected 20000 VND per started hour plus violations.

122. Verify list endpoints after each flow: buildings, floors, slots, monthly passes, active sessions, slot registrations.

## 16. Current Limitations To Remember

- Temporary QR/session card pool routes are not implemented yet.
- QR digital pass generation/validation route is not implemented yet.
- Violation CRUD table/API is not implemented yet; checkout accepts violationFee directly.
- Package plan/pricing policy CRUD is not implemented yet; fees are hard-coded: motorbike 4000/turn, car 20000/hour.
- Reports endpoints are not implemented yet.
- VNPay live testing requires real sandbox credentials in .env.